Paralel Programlama

Dr.Öğ.Üyesi Ömer ÇETİN

Ömer ÇETİN

Bu ders yansıları Dr.Öğ.Üyesi Ömer ÇETİN tarafından ders meteryali olarak oluşturulmuş ve tamamının ya da bir kısmının kullanımı kendi iznine bağlıdır. İzin alınmadan kullanılması ve paylaşılması yasaktır. Tüm hakları saklıdır. Copyrights © 2020
omer_cetin@Outlook.com.tr

Paralel Programlama

KONU - 7

Ömer ÇETİN

İşlem ve İplik

- İşlem (process), yürütülmekte olan bir bilgisayar programının bir örneğidir. Program kodunu ve geçerli etkinliğini içerir.
- Bir işletim sistemi tarafından programlanabilen en küçük işlem birimi ipliktir (thread).
- İplikler (thread) ve işlemler (process) arasındaki farklar:
- – Bir işlemin içinde çok sayıda (en az bir) iplik bulunur. Aynı işlem içinde birden fazla iş parçacığı bulunabilir ve bellek gibi kaynakları paylaşabilir. Bir işlemin iş parçacığı, ikincisinin talimatlarını (kodunu) ve içeriğini (değişkenlerinin herhangi bir anda referans aldığı değerler) paylaşır.
- –Farklı işlemler bu kaynakları paylaşmaz.

Ömer ÇETİN

OpenMP Nedir?

- **OpenMP**, paylaşımlı bellekli çoklu işlemcili mimariler için geliştirilmiş ve derleyici direktifleri yardımıyla paralel programlama yapan bir uygulama geliştirme ara yüzüdür (API).
- Derleyicide derlenen program komutlarını paralel çoklu işlemciye/çekirdeğe sahip sistemlerde dağıtır ve paralel olarak işletilmesinin sağlar.
- OpenMP uygulamaları C/C++ ve Fortran dilleri ile geliştirilebilir ve bu dillere ait birçok derleyici OpenMP desteği vermektedir.

Ömer ÇETİN

OpenMP Nedir?

- OpenMP, orijinal seri kod için müdahaleci bir yapıda değildir: komutlar fortran ve C / C ++ için programlama dilleri için yorum ifadelerinde görünür.

- OpenMP web sitesi:
  www.openmp.org

Ömer ÇETİN

OpenMP Nedir?

- Paylaşılan hafıza ve iş parçacığı-temelli (thread-based) paralellik sunar;
  - OpenMP, paylaşılan bellek programlama paradigmasındaki çoklu iş parçacıklarının varlığına dayanır.
  - Bir paylaşılan bellek işlemi birden fazla iplikten oluşur.
- Açık Paralellik;
  - Programcı paralelleştirme üzerinde tam kontrol sahibidir. OpenMP otomatik bir paralel programlama modeli değildir.
- Derleyici direktifine dayalıdır;
  - Çoğu OpenMP paralelliği, kaynak koduna gömülü derleyici yönergelerinin kullanılmasıyla belirlenir.

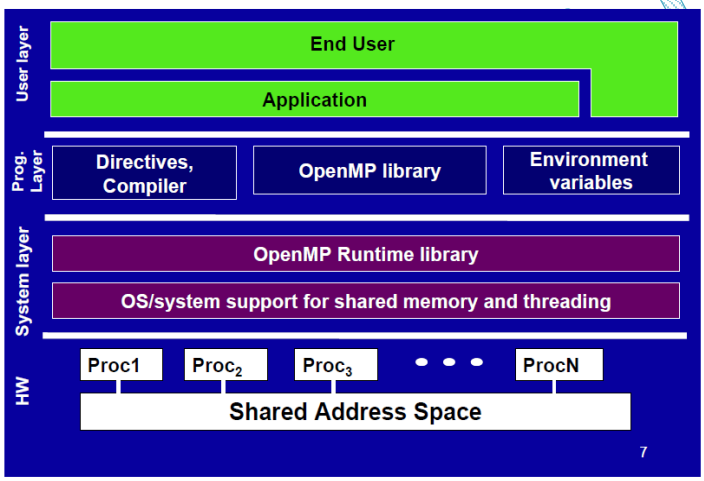
Ömer ÇETİN

OpenMP Nedir?

- OpenMP programlama modelinin sahip olduğu bazı **temel özellikler** şöyledir:
  - İş parçacığı tabanlı paralellik
  - Dallan-Birleş (Fork-Join) modeli
  - İç içe döngüleri paralel yapabilme
  - Dinamik iş parçacıkları
  - Derleyici direktifleri yardımıyla paralel programlama

Ömer ÇETİN

OpenMP Nedir?

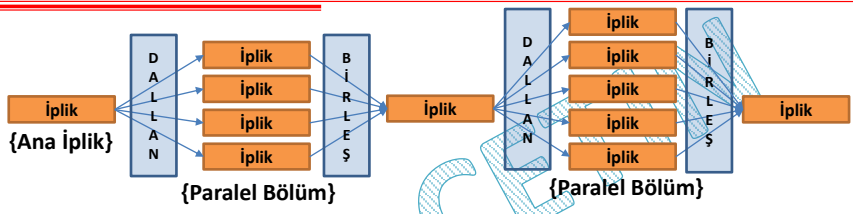


OpenMP Nedir?

- **OpenMP taşınabilir**: HP, IBM, Intel, SGI, SUN ve diğerleri tarafından destekleniyor...
  - Paylaşılan bellek programlarını yazmak için fiilen bir standarttır.
- OpenMP art arda uygulanabilir; tek bir işleve veya tek seferde bir döngünün tamamına...
- Sıralı (sequential) bir programdan paralel program geliştirmek için güzel bir yol...

Ömer ÇETİN

OpenMP ile Paralel Threadler



- OpenMP, paralel yürütme amacıyla **fork-join** modelini kullanır.
  - Tüm OpenMP programları tek bir ana iplik (thread) ile başlar.
  - Ana thread, paralel iş parçacığı takımını dallanma (FORK) ile oluşturduğunda, paralel bir bölge sırayla yürütülür.
  - Çok sayıda thread paralel bölgeyi tamamladığında, sıralı yürütülecek ana iş ipliğine (thread'e) dönülür. Birleş (JOIN) noktasında senkronize edilirler ve paralel işlem son bulur.

Ömer ÇETİN

OpenMP ile Paralel Threadler

Seri Kod Bloğu

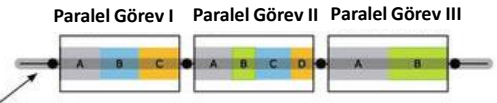
Paralel Kod Bloğu

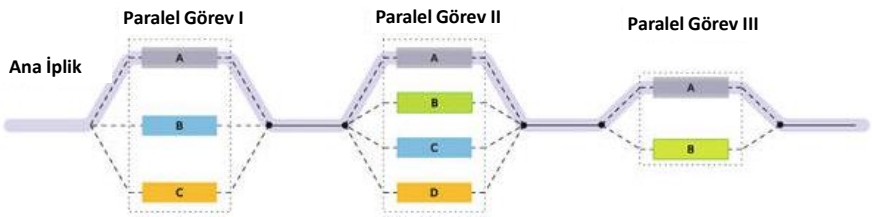
Seri Kod Bloğu

```
#include <omp.h>
main () {
  int var1, var2, var3;
  . . .
     Seri Kod Bloğu
  . . .
  /* Paralel Bölge başlangıcı. Fork ile bir grup threads oluşturulur .Değişken yapıları belirlenir*/
  #pragma omp parallel private(var1, var2) shared(var3)
  {
    /* Tüm threadler tarafından koşturulan paralel kod kısmı */
    . . .
    /* Atüm paralel threadler ana thread üzerinde join işlemi gerçekleştiri r ve sonlanır */
  }
  Seri Kod Bloğu devamı
  . . .
}
```

Ömer ÇETİN

OpenMP ile Paralel Threadler





OpenMP ile Paralel Threadler

- OpenMP
  - Tüm üreticiler tarafından zorunlu olarak **uygulanmamıştır**,
  - Dağıtılmış bellek paralel sistemler için çok amaçlı kullanılamaz (sadece **paylaşılan adrese dayalı sistemler** için tasarlanmıştır),
  - Paylaşılan hafızanın en verimli şekilde kullanılmasını **garanti etmez**,
  - **Veri bağımlılıklarını** (data dependencies), **veri çatışmalarını** (data conflicts), **yarış koşullarını** (race conditions) veya **ölümcül kilitlenmeleri** (deadlocks) kontrol etmek gereklidir,
  - Kod dizilerini kontrol etmek gereklidir,
  - Derleyici tarafından üretilen **otomatik paralelleşmeyi** kapsayacak şekilde çok yönlülük ve bu paralelleşmeye yardımcı olmak için derleyiciye yönergeler sunmaz,
  - Aynı dosyaya giriş veya çıkışın paralel yürütüldüğünde **senkron olacağını** garanti etmez...

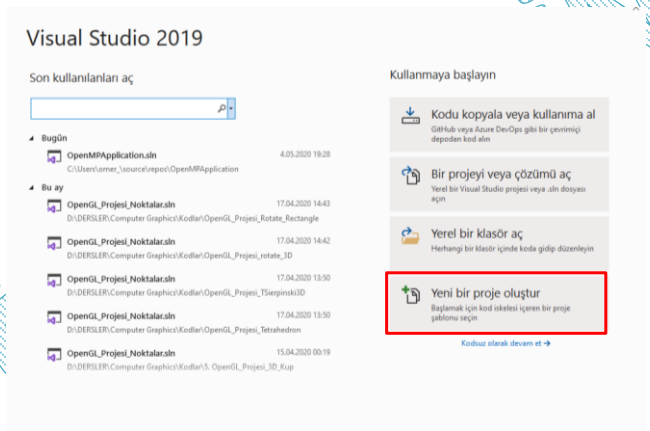
Ömer ÇETİN

OpenMP Derleme

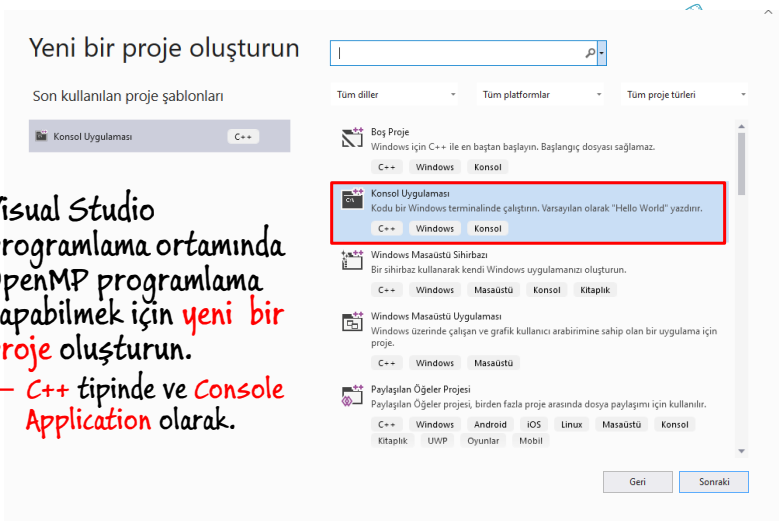
- **Gcc 4.2** ve üzeri sürümler OpenMP 3.0 ve üzeri desteği sunar.
  - gcc –fopenmp deneme.c
- **Microsoft Visual Studio 2010** ve üzeri sürümler, OpenMP 2.0 ve üzeri desteği sunar.
  - https://msdn.microsoft.com/en-us/library/tt15eb9t.aspx

Ömer ÇETİN

Microsoft Visual Studio 2019

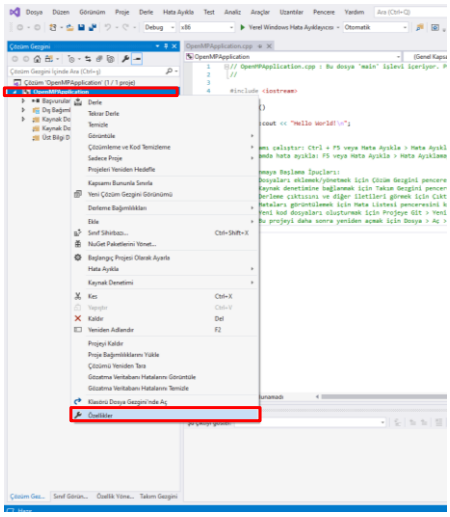


Microsoft Visual Studio 2019



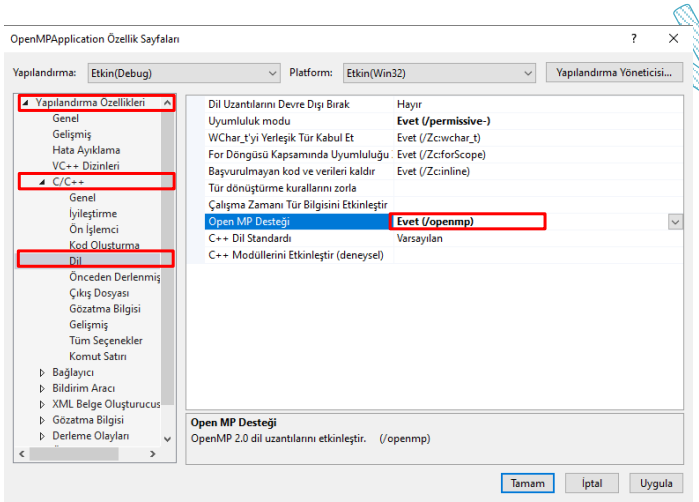
- Visual Studio programlama ortamında OpenMP programlama yapabilmek için yeni bir proje oluşturun.
  - C++ tipinde ve Console Application olarak.

Microsoft Visual Studio 2019

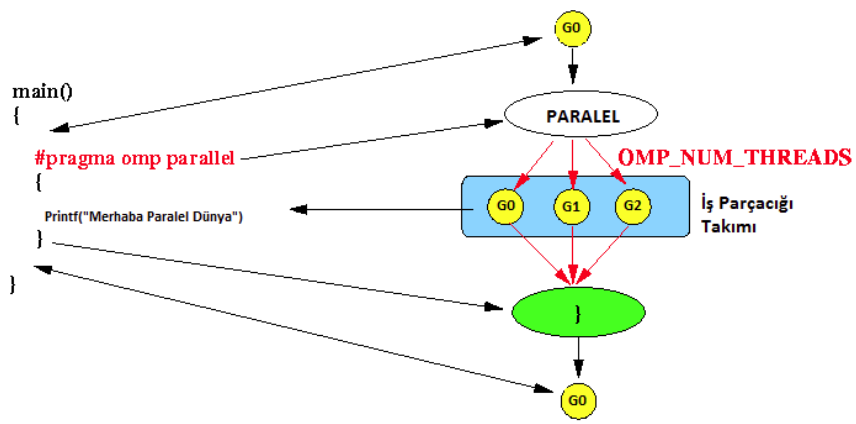


- 1. Projenin Özellik Sayfaları (**Properties**) iletişim kutusunu açın.
- 2. Yapılandırma Özellikleri (**Configuration Properties**) düğümünü genişletin.
- 3. **C / C ++** düğümünü genişletin.
- 4. Dil (**Language**) özelliği sayfasını seçin.
- 5. OpenMP Destek (**OpenMP Support**) değerini değiştirin.

Microsoft Visual Studio 2019



OpenMP HelloWorlds ☺



OpenMP HelloWorlds ☺

- OpenMP kütüphanesini C/C++ programlama dilinde kullanabilmek için programa "**omp.h**" başlık dosyası eklenmelidir.
- Bu dosya OpenMP'nin kullandığı gerekli veri türlerini ve çalışma zamanı kütüphane rutinlerini içerir.

Ömer ÇETİN

OpenMP HelloWorlds ☺

OpenMP işlevlerini kullanmanın üç yolu vardır:

- **OpenMP API bir dizi işlev sunar.** Bunlar sıradan C / C ++ fonksiyonlarına benzerler. Hepsi omp_ ile başlar. Bir fonksiyonun örneği omp_get_thread_num () olup o anki iş parçacığının kimlik numarasını döndürür.
- OpenMP'yi kullanmanın ikinci yolu **çevresel değişkenler**dir. Hepsi OMP_ ile başlar. Örnek olarak, programın kullanabileceği iş parçacığı sayısını belirten OMP_NUM_THREADS verilebilir.
- Son yol **sözde pragmalar** kullanmaktır. Hepsi #pragma omp ile başlar. Bir örnek

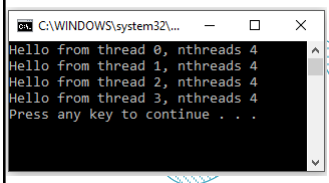
```
#pragma omp parallel
{
    // Yapısal Blok
}
```

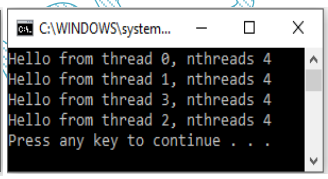
Ömer ÇETİN

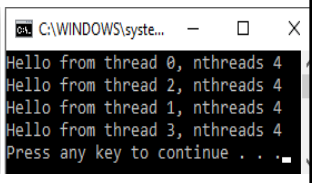
OpenMP HelloWorlds ☺

```
#include <iostream>
#include <omp.h>
#include <stdio.h>

int main()
{
  #pragma omp parallel
  printf("Hello from thread %d, nthreads %d \n", omp_get_thread_num(),  omp_get_num_threads());
}
```

CTRL + F5







Ömer ÇETİN

OpenMP HelloWorlds ☺
Kaç İplik Koşturacak?

```
#include <iostream>
#include <omp.h>
#include <stdio.h>
#include <iostream>

int main()
{
    #pragma omp parallel num_threads(8)
    printf("Hello from thread %d, nthreads %d \n",
omp_get_thread_num(),  omp_get_num_threads());
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 8
Hello from thread 1, nthreads 8
Hello from thread 6, nthreads 8
Hello from thread 3, nthreads 8
Hello from thread 4, nthreads 8
Hello from thread 2, nthreads 8
Hello from thread 5, nthreads 8
Hello from thread 7, nthreads 8
Press any key to continue . . .
```

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>
#include <iostream>

int main()
{
    omp_set_num_threads(5);
    #pragma omp parallel //num_threads(8)
    printf("Hello from thread %d, nthreads %d \n",
omp_get_thread_num(),  omp_get_num_threads());
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 5
Hello from thread 1, nthreads 5
Hello from thread 2, nthreads 5
Hello from thread 3, nthreads 5
Hello from thread 4, nthreads 5
Press any key to continue . . .
```

Ömer Çetin

OpenMP HelloWorlds ☺
Kaç İplik Koşturacak?

OMP_NUM_THREADS

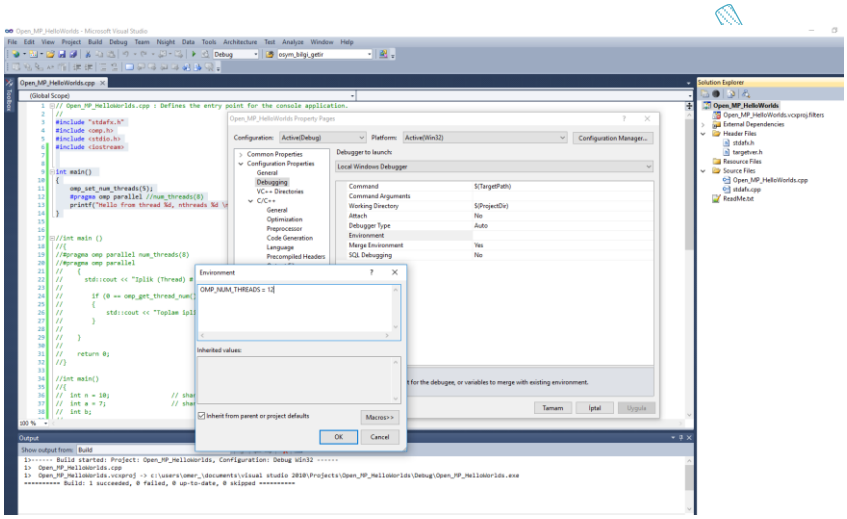
- MS Visual Studio ortamında OpenMP programlamada koşan paralel thread sayısını değiştirmek için, **Solution Explorer** penceresinde projenize sağ tıklayın ve özelliklerini seçin. Sol sütunda " **Debugging** " yı seçin. Ortam değişkenlerini ayarlayabileceğimiz " **Environment** " adlı bir alan var.
- Bunu ortam değişkenlerine eklemeyi deneyin:

"**OMP_NUM_THREADS = 4**"

ve programınızın dört thread ile koşacağını kontrol edin.

Ömer Çetin

OpenMP HelloWorlds ☺
Kaç İplik Koşturacak?




```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>
#include <iostream>


int main()
{
  const int size=24;
  int sonTable[size];
  omp_set_dynamic(0); // C++ ile derleme icin
  omp_set_num_threads(4);
  #pragma omp parallel for
  for (int n=0; n<size; ++n)
  {
    sonTable[n]=n*2;
    printf ("Table %d elemani =%d from thread %d,
nthreads %d \n", n, sonTable[n], omp_get_thread_num(),
omp_get_num_threads());
  }
}
```

C:\WINDOWS\system32\cmd.exe

```
Table 0 elemani =0 from thread 0, nthreads 4
Table 12 elemani =24 from thread 2, nthreads 4
Table 18 elemani =36 from thread 3, nthreads 4
Table 19 elemani =38 from thread 3, nthreads 4
Table 20 elemani =40 from thread 3, nthreads 4
Table 21 elemani =42 from thread 3, nthreads 4
Table 22 elemani =44 from thread 3, nthreads 4
Table 23 elemani =46 from thread 3, nthreads 4
Table 1 elemani =2 from thread 0, nthreads 4
Table 2 elemani =4 from thread 0, nthreads 4
Table 3 elemani =6 from thread 0, nthreads 4
Table 4 elemani =8 from thread 0, nthreads 4
Table 5 elemani =10 from thread 0, nthreads 4
Table 6 elemani =12 from thread 1, nthreads 4
Table 7 elemani =14 from thread 1, nthreads 4
Table 8 elemani =16 from thread 1, nthreads 4
Table 9 elemani =18 from thread 1, nthreads 4
Table 10 elemani =20 from thread 1, nthreads 4
Table 13 elemani =26 from thread 2, nthreads 4
Table 14 elemani =28 from thread 2, nthreads 4
Table 15 elemani =30 from thread 2, nthreads 4
Table 16 elemani =32 from thread 2, nthreads 4
Table 17 elemani =34 from thread 2, nthreads 4
Table 11 elemani =22 from thread 1, nthreads 4
Press any key to continue . . .
```

Ömer Ç

OpenMP HelloWorlds ☺
Kaç İplik Koşturacak?

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>
#include <iostream>


int main()
{
  const int size=24;
  int sonTable[size];
  omp_set_dynamic(0); // C++ ile derleme için
  omp_set_num_threads(12);
  #pragma omp parallel for
  for (int n=0; n<size; ++n)
  {
    sonTable[n]=n*2;
    printf ("Table %d elemani =%d from thread %d,
nthreads %d \n", n, sonTable[n], omp_get_thread_num(),
omp_get_num_threads());
  }
}
```

C:\WINDOWS\system32\cmd.exe

```
Table 0 elemani =0 from thread 0, nthreads 12
Table 1 elemani =2 from thread 0, nthreads 12
Table 4 elemani =8 from thread 2, nthreads 12
Table 5 elemani =10 from thread 2, nthreads 12
Table 2 elemani =4 from thread 1, nthreads 12
Table 3 elemani =6 from thread 1, nthreads 12
Table 14 elemani =28 from thread 7, nthreads 12
Table 16 elemani =32 from thread 8, nthreads 12
Table 10 elemani =20 from thread 5, nthreads 12
Table 11 elemani =22 from thread 5, nthreads 12
Table 6 elemani =12 from thread 3, nthreads 12
Table 7 elemani =14 from thread 3, nthreads 12
Table 20 elemani =40 from thread 10, nthreads 12
Table 21 elemani =42 from thread 10, nthreads 12
Table 12 elemani =24 from thread 6, nthreads 12
Table 13 elemani =26 from thread 6, nthreads 12
Table 17 elemani =34 from thread 8, nthreads 12
Table 8 elemani =16 from thread 4, nthreads 12
Table 9 elemani =18 from thread 4, nthreads 12
Table 15 elemani =30 from thread 7, nthreads 12
Table 22 elemani =44 from thread 11, nthreads 12
Table 23 elemani =46 from thread 11, nthreads 12
Table 18 elemani =36 from thread 9, nthreads 12
Table 19 elemani =38 from thread 9, nthreads 12
Press any key to continue . . .
```


Kim Önce Yazar ?

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>
#include <iostream>

int main ()
{

#pragma omp parallel
    {
std::cout << "Iplik (Thread) # = " << omp_get_thread_num() << std::endl;

        if (0 == omp_get_thread_num())
        {
            std::cout << "Toplam iplik Sayisi = "<< omp_get_num_threads() << std::endl;
        }

    }

    return 0;
}
```

Beklenen Program Çıktısı

```
Iplik (thread) # = 0
Toplam iplik Sayisi = 4
Iplik (thread) # = 1
Iplik (thread) # = 3
Iplik (thread) # = 2
```

Gerçekleşen Program Çıktısı

C:\WINDOWS\system32\cmd.exe

```
Iplik (Thread) # = Iplik (Thread) # = 1
0
Toplam iplik Sayisi = 4
Iplik (Thread) # = 3
Iplik (Thread) # = 2
Press any key to continue . . .
```

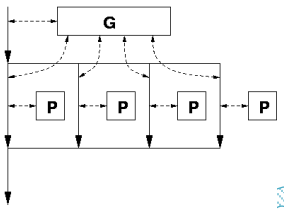
Kim Önce Hesaplar ?

```
C:\WINDOWS\system32\cmd.exe
Table 158 elemani = 316 from thread 2, nthreads 4
Table 159 elemani = 318 from thread 2, nthreads 4
Table 160 elemani = 320 from thread 2, nthreads 4
Table 161 elemani = 322 from thread 2, nthreads 4
Table 162 elemani = 324 from thread 2, nthreads 4
Table 163 elemani = 326 from thread 2, nthreads 4
Table 164 elemani = 328 from thread 2, nthreads 4
Table 48 elemani = 96 from thread 0, nthreads 4
Table 49 elemani = 98 from thread 0, nthreads 4
Table 50 elemani = 100 from thread 0, nthreads 4
Table 51 elemani = 102 from thread 0, nthreads 4
Table 52 elemani = 104 from thread 0, nthreads 4
Table 53 elemani = 106 from thread 0, nthreads 4
Table 54 elemani = 108 from thread 0, nthreads 4
Table 55 elemani = 110 from thread 0, nthreads 4
Table 56 elemani = 112 from thread 0, nthreads 4
Table 57 elemani = 114 from thread 0, nthreads 4
Table 58 elemani = 116 from thread 0, nthreads 4
Table 59 elemani = 118 from thread 0, nthreads 4
Table 60 elemani = 120 from thread 0, nthreads 4
Table 61 elemani = 122 from thread 0, nthreads 4
Table 62 elemani = 124 from thread 0, nthreads 4
Table 63 elemani = 126 from thread 0, nthreads 4
Table 165 elemani = 330 from thread 2, nthreads 4
Table 166 elemani = 332 from thread 2, nthreads 4
Table 167 elemani = 334 from thread 2, nthreads 4
Table 168 elemani = 336 from thread 2, nthreads 4
Table 169 elemani = 338 from thread 2, nthreads 4
Table 170 elemani = 340 from thread 2, nthreads 4
Table 171 elemani = 342 from thread 2, nthreads 4
```

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
  const int size = 256;
    int sonTable[size];

    #pragma omp parallel for
    for(int n=0; n<size; ++n)
      {
        sonTable[n] = (2 * n );
        printf("Table %d elemani = %d
          from thread %d, nthreads %d
          \n", n, sonTable[n],
          omp_get_thread_num(),
          omp_get_num_threads());
      }
}
```

Ö. ÇETİN

OpenMP Veri Modeli



P = private data space
G = global data space

- Özel ve paylaşılan değişkenler
  - Genel veri alanındaki değişkenlere tüm paralel threadler (**shared - paylaşılan değişkenler**) erişir.
  - Bir ipliğin özel alanındaki değişkenlere yalnızca thread tarafından erişilebilir (**private - özel değişkenler**)
    - ilk değerlere ve sonuçların bölgenin dışına kopyalanmasına bağlı olarak çeşitli değişiklikler gösterebilirler.

Ö. ÇETİN

OpenMP Veri Modeli

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
    int x = 5;
    #pragma omp parallel
    {
        printf("Hello from thread %d, nthreads %d, hesaplanan: %d \n", omp_get_thread_num(),
omp_get_num_threads(), x);
        x = x+1;
    }
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 5
Hello from thread 1, nthreads 4, hesaplanan: 5
Hello from thread 3, nthreads 4, hesaplanan: 5
Hello from thread 2, nthreads 4, hesaplanan: 5
Press any key to continue . . .
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 5
Hello from thread 1, nthreads 4, hesaplanan: 5
Hello from thread 2, nthreads 4, hesaplanan: 5
Hello from thread 3, nthreads 4, hesaplanan: 7
Press any key to continue . . .
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 5
Hello from thread 1, nthreads 4, hesaplanan: 6
Hello from thread 2, nthreads 4, hesaplanan: 6
Hello from thread 3, nthreads 4, hesaplanan: 6
Press any key to continue . . .
```

Ömer ÇETİN

OpenMP Veri Modeli

Paylaşımlı (Shared) Değişkenler

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
    int i = 0;
    int n = 10;
    int a = 7;

    #pragma omp parallel for
    for (i = 0; i < n; i++){
        int b = a + i;
        printf("Hello from thread %d, nthreads %d,
hesaplanan: %d \n", omp_get_thread_num(),
omp_get_num_threads(), b);
    }
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 3, nthreads 4, hesaplanan: 16
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 2, nthreads 4, hesaplanan: 14
Hello from thread 1, nthreads 4, hesaplanan: 12
Press any key to continue . . .
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 2, nthreads 4, hesaplanan: 14
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 1, nthreads 4, hesaplanan: 12
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 3, nthreads 4, hesaplanan: 16
Press any key to continue . . .
```

Ömer ÇETİN

OpenMP Veri Modeli

Paylaşımlı (Shared) ve Özel (Private) Değişkenler

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
    int n = 10;  // shared
    int a = 7;   // shared

    #pragma omp parallel for
    for (int i = 0; i < n; i++) // i private
    {
    int b = a + i;              // b private
    printf("Hello from thread %d, nthreads %d,
hesaplanan: %d \n", omp_get_thread_num(),
omp_get_num_threads(), b);
    }
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 2, nthreads 4, hesaplanan: 14
Hello from thread 3, nthreads 4, hesaplanan: 16
Hello from thread 1, nthreads 4, hesaplanan: 12
Press any key to continue . . .
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 1, nthreads 4, hesaplanan: 12
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 3, nthreads 4, hesaplanan: 16
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 2, nthreads 4, hesaplanan: 14
Press any key to continue . . .
```


OpenMP Veri Modeli

Paylaşımlı (Shared) ve Özel (Private) Değişkenler

```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
    int n = 10;                 // shared
    int a = 7;                  // shared

    #pragma omp parallel for shared(n, a)
    for (int i = 0; i < n; i++)
    {
        int b = a + i;
        printf("Hello from thread %d, nthreads
%d, hesaplanan: %d \n", omp_get_thread_num(),
omp_get_num_threads(), b);
    }
}
```

C:\WINDOWS\system32\cmd.exe

```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 2, nthreads 4, hesaplanan: 14
Hello from thread 3, nthreads 4, hesaplanan: 16
Hello from thread 1, nthreads 4, hesaplanan: 12
Press any key to continue . . .
```

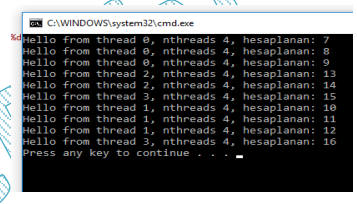
C:\WINDOWS\system32\cmd.exe

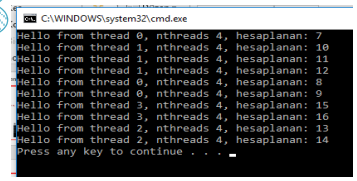
```
Hello from thread 0, nthreads 4, hesaplanan: 7
Hello from thread 2, nthreads 4, hesaplanan: 13
Hello from thread 2, nthreads 4, hesaplanan: 14
Hello from thread 1, nthreads 4, hesaplanan: 10
Hello from thread 1, nthreads 4, hesaplanan: 11
Hello from thread 1, nthreads 4, hesaplanan: 12
Hello from thread 0, nthreads 4, hesaplanan: 8
Hello from thread 0, nthreads 4, hesaplanan: 9
Hello from thread 3, nthreads 4, hesaplanan: 15
Hello from thread 3, nthreads 4, hesaplanan: 16
Press any key to continue . . .
```

OpenMP Veri Modeli

Paylaşımlı (Shared) Değişkenler

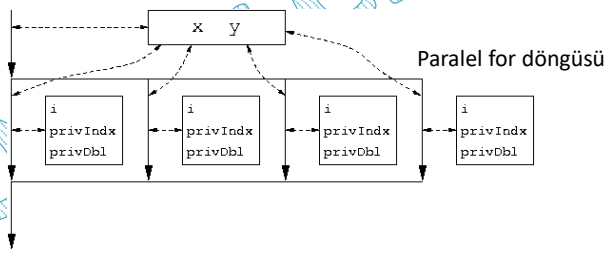
```
#include "stdafx.h"
#include <omp.h>
#include <stdio.h>

int main()
{
    int n = 10;                 // shared
    int a = 7;                  // shared
    int b;

    #pragma omp parallel for shared(n, a) private(b)
    for (int i = 0; i < n; i++)
    {
        b = a + i;
        printf("Hello from thread %d, nthreads %d,
hesaplanan: %d \n", omp_get_thread_num(),
omp_get_num_threads(), b);
    }
}
```





OpenMP Veri Modeli

```
#pragma omp parallel for private( privIndx, privDbl )
for ( i = 0; i < arraySize; i++ ) {
    for ( privIndx = 0; privIndx < 16; privIndx++ ) { privDbl = ( (double) privIndx ) / 16;
    y[i] = sin( exp( cos( - exp( sin(x[i]) ) ) ) ) + cos( privDbl );
  }
}
```



execution context for "arrayUpdate_II"

OpenMP Komut Modeli

- **Format:**

  #progma omp directive-name [clause,..]

  (çok sayıda satır için '\' ifadesi kullanılır ...)

- **Örnek:**

  #pragma omp parallel default(shared) private(beta,pi)

- **Komutun geçerliliği sadece bir blok içindir.**

  { ... } arasında.

Ömer ÇETİN

OpenMP Komut Modeli

- Birden çok thread tarafından yürütülecek bir kod bloğu.

#pragma omp parallel *[clause …]*

{

……

} (implied barrier)

Clauses: if (expression), private *(list),* shared *(list),* default (shared | none), reduction *(operator: list),* firstprivate*(list),* lastprivate*(list)*

- **if (expression):** (Sıfırdan farklı) olarak değerlendirilirse, paralel olarak yürütmek için paralel bölge kodu neden olan bir tamsayı ifade. ifade yanlış (sıfır) olarak değerlendirilirse, paralel bölge (tek bir iş parçacığı tarafından) seri içinde yürütülür.
- **private(list):** Bu tip tanımlanan değişkenler özel ve yereldir (blok dışında yer alan değişkenler il ilişkisi yoktur.)
- **shared(list):** Paralel tüm thread'ler tarafından erişilebilen paylaşılan değişkenleri tanımlar.
- **default (none|shared)**

Ömer ÇETİN

OpenMP Komut Modeli

```
#pragma omp parallel private(i,x) reduction(+:toplam)
{
    #pragma omp for
    for(i=0; i<adim_sayisi; i++)
        {
                x = (i+0.5)*adim_miktari;
                toplam += 4.0/(1.0 + x * x);
        }
}
```

#pragma omp parallel direktifi: Paralel hesaplama yapılacak bir alan oluşturulur. Bu alan içindeki tüm komutlar paralel olarak koşulur.

Ömer ÇETİN

OpenMP Komut Modeli

```
#pragma omp parallel private(i,x) reduction(+:toplam)
{
    #pragma omp for
    for(i=0; i<adim_sayisi; i++)
        {
                x = (i+0.5)*adim_miktari;
                toplam += 4.0/(1.0 + x * x);
        }
}
```

- **private(i,x)** direktifi: *For* döngüsünde kullanılacak *i* indis değeri ve ara bir hesaplama değerinin tutulacağı *x* değişkeni özel (private) olarak tanımlanır. Paralel olarak oluşturulacak her iş parçacığı *i* ve *x* değişkenlerini yerel (olarak) oluşturacak ve sadece kendileri kullanacak.

Ömer ÇETİN

OpenMP Komut Modeli

```
#pragma omp parallel private(i,x) reduction(+:toplam)
{
    #pragma omp for
    for(i=0; i<adim_sayisi; i++)
        {
                x = (i+0.5)*adim_miktari;
                toplam += 4.0/(1.0 + x * x);
        }
}
```

- reduction(+:toplam) direktifi: Her bir iş parçacığı kendi iterasyon aralığındaki hesaplama değerlerini kendi yerel toplam değişkenlerinde toplar ve tüm iş parçacıkları hesaplamalarını bitirdiklerinde reduction işlemi ile her iş parçacığından gelen toplam değerleri toplanır ve parçalanan görevler birleştirilir.

OpenMP Komut Modeli

```
sum = 0.0;
#pragma parallel default(none) shared (n, x) private (i) reduction(+ : sum)
{
  for (i=0; i<n; i++)
    sum = sum + x(i);
}
```

- sum değerinin güncellenmesinde yarış durumunun oluşmasından kaçınılmalı.
- Reduction değişken tanımlaması ile OpenMP yarış durumunu engelleyecek şekilde bir yapı oluşturur.

- Firstprivate(list): değişkenler, bloğa girmeden önceki değerle başlatılır
- Lastprivate(list): değişkenler bloktan çıkıp güncellenir.

Ömer ÇETİN

OpenMP Komut Modeli

```
#pragma omp parallel private(i,x) reduction(+:toplam)
{
    #pragma omp for
    for(i=0; i<adim_sayisi; i++)
        {
                x = (i+0.5)*adim_miktari;
                toplam += 4.0/(1.0 + x * x);
        }
}
```

- #pragma omp for direktifi: Bu direktif ile döngüdeki toplam iterasyon her thread'e eşit miktarda dağıtılır ve döngünün her parçasının farklı bir thread tarafından hesaplanması sağlanır.

Ömer ÇETİN

OpenMP Komut Modeli

- #pragma omp for [clause …]
- #pragma omp section [clause …]
- #pragma omp single [clause …]

- İş, çok sayıda paralel thread üzerine dağıtılır.
- Paralel bölge içine alınmalıdır.
- Girişte senkronizasyon yok, çıkışta standart senkronizasyon var (aksi belirtilmedikçe)

Ömer ÇETİN

OpenMP Komut Modeli

```
#pragma omp parallel default(none)\
        shared(n,a,b,c,d) private(i)
  {
    #pragma omp sections nowait
    {
      #pragma omp section
        for (i=0; i<n-1; i++)
            b[i] = (a[i] + a[i+1])/2;

      #pragma omp section
        for (i=0; i<n; i++)
            d[i] = 1.0/c[i];

    } /*-- End of sections --*/

  } /*-- End of parallel region --*/
```


OpenMP Komut Modeli

```
#pragma omp parallel
#pragma omp for
    for(. . .)
```

→

```
#pragma omp parallel for
For(. . .)
```

Single Paralel Döngü

```
#pragma omp parallel
#pragma omp sections
 {. . .}
```

→

```
#pragma omp parallel sections
{. . .}
```

Single Paralel Bölüm

Senkronizasyon: barrier

```
for (I=0; I<N; I++)
   a[I] = b[I] + c[I];

for (I=0; I<N; I++)
   d[I] = a[I] + b[I]
```

Her iki döngü de paralel olarak koşturulacak...
İki döngü arasında bir senkronizasyon noktası yok...
Problem Nedir ?

```
for(I=0; I<N; I++)
   a[I] = b[I] + c[I];

#pragma omp barrier

for(I=0; I<N; I++)
   d[I] = a[I] + b[I]
```

Ömer ÇETİN

Kaçınılmaz Senkronizasyon

```
01 #pragma omp parallel shared (A, B, C)
02 {
03   Calculationfunction(A,B);
04   printf("B was calculated from A\n");
05   #pragma omp barrier
06   Calculationfunction(B,C);
07   printf("C was calculated from B\n");
08 }
```

Bu koddaki Calculationfunction () satırı, birincisine atıfta bulunarak ikinci argümanı hesaplar. Bu durumda argümanlar diziler olabilir ve hesaplama işlevi karmaşık bir matematiksel matris işlemi olabilir.
Burada, #pragma omp barrier'ı kullanmak çok önemlidir - bunun başarısız olması, B'deki hesaplama için değerler bulunmadan önce bazı threadlerin hesaplamaları ikinci turuyla başlamasını gerektirir.
Bazı OpenMP yapıları (parallel, for, single gibi) nowait için #pragma omp'de olduğu gibi nowait yan tümcesi ekleyerek açıkça devre dışı bırakabileceğiniz gizli bir senkronizasyon içerir.

Diğer eşitleme mekanizmaları şunları içerir:

#pragma omp master {Code}: Sadece bir kez ve yalnızca ana iş parçacığı tarafından çalıştırılan kod.

#pragma omp single {Code}: Sadece bir kez çalıştırılan kod, ancak mutlaka ana thread tarafından değil

#pragma omp flush (Değişkenler): Ana belleğe geri yazılmış önbellek değişkenleri, bellekte tutarlı bir görünüm sağlar.

Ömer ÇETİN

Kritik Bölge

```
for(I=0; I<N; I++)
{
  ......
  sum += A[I];
  ......
}
```

sum değişkeni paylaşılıyor ise döngü paralel olarak koşturulabilir mi?

```
for(I=0; I<N; I++)
{
  ......
  #pragma omp critical
  {
    sum += A[I];
  }
  ......
}
```

Ömer ÇETİN

Yarış Durumundan Kaçınma

```
01  #ifdef _OPENMP
02  #include <omp.h>
03  #endif
04  #include <stdio.h>
05  int main() {
06      double a[1000000];
07      int i;
08      #pragma omp parallel for
09      for (i=0; i<1000000; i++) a[i]=i;
10      double sum = 0;
11      #pragma omp parallel for shared (sum) private (i)
12      for ( i=0; i < 1000000; i++) {
13          #pragma omp critical (sum)
14          sum = sum + a[i];
15      }
16      printf("sum=%lf\n",sum);
17  }
```

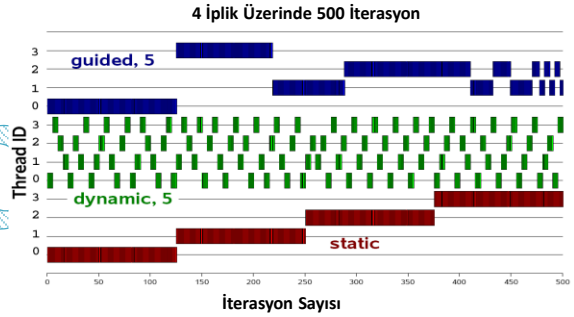
Satır 13'te OpenMP #pragma omp critical (sum_total) deyimi olmadan yarış durumu ortaya çıkabilir...

Ömer ÇETİN

Planlama

- OMP_SCHEDULE
- Planlama (döngülerin paralel olarak nasıl yürütüleceğine karar verecek – sadece döngülerde geçerlidir):

schedule (static | dynamic | guided [, chunk])



Planlama

- schedule (static | dynamic | guided | Runtime [, chunk])
- Schedule (tür [,yığın_boyutu])
- Yığın (chunk) iterasyon alanının bitişik, boş olmayan bir alt kümesidir. Sabit tam sayı olarak tanımlanır.
- Dört tür planlama türü vardır:

Ömer ÇETİN

Planlama

– **Static:** İterasyonlar, yığın boyutu boyutunda parçalara ayrılır. Parçalar, iplik sırasına göre dairesel ve rotasyonel bir şekilde ipliklere atanır. Atanacak son yığın belirlenenden az miktarda olabilir. Bir yığın boyutu belirtilmediğinde İterasyon sayısı yaklaşık eşit boyuttaki parçalara bölünür.

Ör

```
schedule(static):
****************
                ****************
                                ****************
                                                ****************
```

```
schedule(static, 4):
****            ****            ****            ****
    ****            ****            ****            ****
        ****            ****            ****            ****
            ****            ****            ****            ****
```

```
schedule(static, 8):
********                        ********
        ********                        ********
                ********                        ********
                        ********                        ********
```


Planlama

– **Dynamic:** İterasyonlar, iplikler onları talep ettikçe atanır. İplik yığın boyutu olarak ifade edilen sayıda iterasyonu yürütür, ardından daha fazla yığın bulana kadar talep ederek yürütmeye devam eder. Son yığın belirlenenden az miktarda olabilir. Bir yığın boyutu belirtilmediğinde varsayılan değer 1 olur.

Ör

```
schedule(dynamic):
*   ** **  * * *  *     *  *    **   *  *  * *      *  *   *
  *      *    *    * *    * *   *    *       * *   *    *
 *      *    *    * *   *   *    *  *      *  *  *  *  *   *
   *  *    *   * *    *  *   *    *   ** *  *   *    *   *
```

```
schedule(dynamic, 1):
    *    *    *       *   *    * *  *  *        *  * *  * *
*  *  *   * *    *  * * *   * *     *   ***  *   *        *
 *   *  *  *  *   ** *    *      *  *  * *   *  *   *   *
  *    *    *  **       *  * *   *        *  *    *  * *  *
```

```
schedule(dynamic, 4):
            ****                ****                ****
****            ****    ****            ****        ****
    ****            ****    ****            ****        ****
        ****                    ****            ****
```

```
schedule(dynamic, 8):
                ********                                ********
                        ********        ********
********                        ********        ********
        ********
```

Planlama

– **Guided**: Dinamik programlama tipine benzer. OpenMP iterasyonları yığın boyutuna böler. İplik yığın boyutu olarak ifade edilen sayıda iterasyonu yürütür, ardından daha fazla yığın bulana kadar talep ederek yürütmeye devam eder. **Dinamik programlama tipiyle olan fark, parça boyutundadır. Bir yığın boyutu, thread sayısına bölünen atanmamış iterasyonların sayısıyla orantılıdır.** Bu nedenle yığınların boyutu azalır.

```
schedule(guided):
                    *********                          *
            ************                   *******  ***
                            *******                     *
****************                    *****      **    *
```

```
schedule(guided, 2):
            ************                   ****     **
                            *******        ***    **
                    *********
****************                    *****      **    **
```

```
schedule(guided, 4):
                            *******
            ************                   ****    ****
                    *********
****************                    *****    ****    ***
```

```
schedule(guided, 8):
            ************                *********       ***
****************
                            ********
                    *********               ********
```

Ömer

Planlama

– **Runtime:** Çalışma zamanı zamanlama türü, çalışma zamanına kadar zamanlama hakkındaki kararı savunur. Bu durumda, zamanlama OMP_SCHEDULE tipi runtime seçilir ve omp_set_schedule fonksiyonu ile koşturulma esnasında açıklanan yöntemlerden birisi olacak şekilde (static | dynamic | guided) koşullara bağlı olarak karar verilerek atanır.

Ömer ÇETİN

Planlama

```
#include <windows.h>
#include <stdio.h>
#include <omp.h>

#define NUM_THREADS 4
#define STATIC_CHUNK 5
#define DYNAMIC_CHUNK 5
#define NUM_LOOPS 20
#define SLEEP_EVERY_N 3

int main()
{
   int nStatic1[NUM_LOOPS], nStaticN[NUM_LOOPS];
   int nDynamic1[NUM_LOOPS], nDynamicN[NUM_LOOPS];
   int nGuided[NUM_LOOPS];
```

Ömer ÇETİN

Planlama

```
omp_set_num_threads(NUM_THREADS);

#pragma omp parallel
{
   #pragma omp for schedule(static, 1)
   for (int i = 0 ; i < NUM_LOOPS ; ++i)
   {
      if ((i % SLEEP_EVERY_N) == 0)
         Sleep(0);
      nStatic1[i] = omp_get_thread_num();
   }

   #pragma omp for schedule(static, STATIC_CHUNK)
   for (int i = 0 ; i < NUM_LOOPS ; ++i)
   {
      if ((i % SLEEP_EVERY_N) == 0)
         Sleep(0);
      nStaticN[i] = omp_get_thread_num();
   }
```

```
#define NUM_THREADS 4
#define STATIC_CHUNK 5
#define DYNAMIC_CHUNK 5
#define NUM_LOOPS 20
#define SLEEP_EVERY_N 3
```

static 1	static 5	dynamic 1	dynamic 5	guided
0	0	0	2	1
1	0	3	2	1
2	0	3	2	1
3	0	3	2	1
0	0	2	2	1
1	1	2	3	3
2	1	2	3	3
3	1	0	3	3
0	1	0	3	3
1	1	0	3	2
2	2	1	0	2
3	2	1	0	2
0	2	1	0	3
1	2	2	0	3
2	2	2	0	0
3	3	2	1	0
0	3	3	1	1
1	3	3	1	1
2	3	3	1	1
3	3	0	1	3

Ömer ÇETİN

Planlama

```
#pragma omp for schedule(dynamic, 1)
for (int i = 0 ; i < NUM_LOOPS ; ++i)
{
  if ((i % SLEEP_EVERY_N) == 0)
    Sleep(0);
  nDynamic1[i] = omp_get_thread_num();
}
```

```
#pragma omp for schedule(dynamic, DYNAMIC_CHUNK)
for (int i = 0 ; i < NUM_LOOPS ; ++i)
{
  if ((i % SLEEP_EVERY_N) == 0)
    Sleep(0);
  nDynamicN[i] = omp_get_thread_num();
}
```

```
#define NUM_THREADS 4
#define STATIC_CHUNK 5
#define DYNAMIC_CHUNK 5
#define NUM_LOOPS 20
#define SLEEP_EVERY_N 3
```

static 1	static 5	dynamic 1	dynamic 5	guided
0	0	0	2	1
1	0	3	2	1
2	0	3	2	1
3	0	3	2	1
0	0	2	2	1
1	1	2	3	3
2	1	2	3	3
3	1	0	3	3
0	1	0	3	3
1	1	0	3	2
2	2	1	0	2
3	2	1	0	2
0	2	1	0	3
1	2	2	0	3
2	2	2	0	0
3	3	2	1	0
0	3	3	1	1
1	3	3	1	1
2	3	3	1	1
3	3	0	1	3

Planlama

```
#pragma omp for schedule(guided)
for (int i = 0 ; i < NUM_LOOPS ; ++i)
{
  if ((i % SLEEP_EVERY_N) == 0)
    Sleep(0);
  nGuided[i] = omp_get_thread_num();
}
}

printf_s("-----------------------------------------------\n");
printf_s("| static | static | dynamic | dynamic | guided |\n");
printf_s("|  %  |  %d  |  %  |  %d  |     |\n",
    STATIC_CHUNK, DYNAMIC_CHUNK);
printf_s("-----------------------------------------------\n");

for (int i=0; i<NUM_LOOPS; ++i)
{
  printf_s("|  %d  |  %d  |  %d  |  %d  |  %d  |\n",
      nStatic1[i], nStaticN[i],
      nDynamic1[i], nDynamicN[i], nGuided[i]);
}
printf_s("-----------------------------------------------\n");
}
```

```
#define NUM_THREADS 4
#define STATIC_CHUNK 5
#define DYNAMIC_CHUNK 5
#define NUM_LOOPS 20
#define SLEEP_EVERY_N 3
```

static 1	static 5	dynamic 1	dynamic 5	guided
0	0	0	2	1
1	0	3	2	1
2	0	3	2	1
3	0	3	2	1
0	0	2	2	1
1	1	2	3	3
2	1	2	3	3
3	1	0	3	3
0	1	0	3	3
1	1	0	3	2
2	2	1	0	2
3	2	1	0	2
0	2	1	0	3
1	2	2	0	3
2	2	2	0	0
3	3	2	1	0
0	3	3	1	1
1	3	3	1	1
2	3	3	1	1
3	3	0	1	3

Ömer Çetin

OpenMP Runtime Environment

- omp_get_num_threads
- omp_get_thread_num
- omp_in_parallel
- Kilitleme (lock) ile ilgili diğer rutinler
- ……

Ömer ÇETİN

C++ ile Yürütme Zamanı Ölçme

PERFORMANS

Ömer ÇETİN

C / C ++ ile Yürütme Zamanını Yüksek Hassasiyetle Ölçme

- **Yürütme zamanı**: Belirli bir görevin yürütme süresi veya CPU zamanı, o görevi yürüten sistemin bir programın çalıştığı zamanı söyleyebileceği şekilde söyleyebilmesi için harcanan süre olarak tanımlanır.
- Bir programın yürütme süresini ölçmenin birden fazla yolu var, ders kapsamında beş farklı yöntemden bahsedilmiştir.

Ömer ÇETİN

1. time() Fonksiyonun C & C++'da Kullanımı ile Zaman Alma

- ***time()***: *time () işlevi Epoch'tan (1 Ocak 1970) bu yana saniye cinsinden süreyi döndürür.*
- ***Header File***: *"time.h"*
- ***Prototype/Syntax***: *time_t time(time_t *tloc);*
- ***Return Value***: *Başarı durumunda, Epoch'dan beri saniye cinsinden geçen süre döndürülür, hata -1 üzerine döndürülür.*

```
// Süresi ölçülmek istenen örnek fonksiyon
void fun() {
   for (int i=0; i<10; i++)
   {

   }
}

int main() {
   /* Epoch'tan (1 Ocak 1970) bu yana saniye cinsinden süreyi döndürür */
   time_t start, end;

   /*Buna şöyle diyebilirsiniz: start = time (NULL); her iki şekilde de,
Epoch'tan bu yana saniye cinsinden toplam süre içerir.*/
   time(&start);

   // C ve C ++ 'nın G / Ç işlemleri senkronizasyon iptali.
   ios_base::sync_with_stdio(false);

   fun();

   // zamanı durdur.
   time(&end);

   // Program tarafından alınan toplam sürenin hesaplanması.
   double time_taken = double(end - start);
   cout << "Time taken by program is : " << fixed << time_taken <<
setprecision(5);
   cout << " sec " << endl;
   return 0;
}
```

Ömer ÇETİN

2. clock() Fonksiyonun C & C++'da Kullanımı ile Zaman Alma

- **clock()**: clock() program başlatıldığından bu yana geçen saat tik sayısını geri döndürür.
- **Header File**: "time.h"
- **Prototype / Syntax**: clock_t clock(void);
- **Geri Dönüş Değeri**: Başarı durumunda, döndürülen değer, clock_t; Kullanılan saniye sayısını bulmak için CLOCKS_PER_SEC ile bölün. hata -1 üzerine döndürülür.

```
#include <bits/stdc++.h>
using namespace std;

// Süresi ölçülmek istenen örnek fonksiyon
void fun() {
   for (int i=0; i<10; i++) {

   }
}

int main() {
/* clock_t clock (void), programın başlatılmasından bu yana geçen
saat tiklerinin sayısını döndürür. CPU tarafından kullanılan
saniye sayısını bulmak için CLOCKS_PER_SEC'e bölmeniz gerekir.
CLOCKS_PER_SEC tipik 32 bit sistemde 1000000'dür.*/
   clock_t start, end;

   /*Başlangıç saat vuruşu numarasının kaydedilmesi.*/
   start = clock();
   fun();
   // Bitiş saat vuruşu numarasının kaydedilmesi.
   end = clock();

   // Program tarafından alınan toplam sürenin hesaplanması.
   double time_taken = double(end - start)/double(CLOCKS_PER_SEC);
   cout << "Program tarafından alınan zaman: " << fixed <<
time_taken << setprecision(5);
   cout << " sn " << endl;
   return 0;
}
```

Ömer ÇETİN

3. gettimeofday() Fonksiyonun C & C++'da Kullanımı ile Zaman Alma

- **gettimeofday()** : Saat dilimine uygun zaman alma fonksiyonu.
- **Header File** : "sys/time.h".
- **Prototype/Syntax**: int gettimeofday (struct timeval *tv, struct timezone *tz);
- Tv (Zaman değeri) argümanı yapısal bir zaman çizelgesidir ve Epoch'tan bu yana saniye ve mikro saniye sayısını verir.
- **Return Value** : return 0 for success, or -1 for failure

```
#include <bits/stdc++.h>
#include <sys/time.h>
using namespace std;

// Süresi ölçülmek istenen örnek fonksiyon
void fun() {
   for (int i=0; i<10; i++) {

   }
}

int main() {
   struct timeval start, end;

   // zamanı başlat.
   gettimeofday(&start, NULL);

   // C ve C++ 'nın G / Ç işlemleri senkronizasyon iptali.
   ios_base::sync_with_stdio(false);

   fun();

   // zamanı durdur.
   gettimeofday(&end, NULL);

   // Yürütülme süresini hesapla.
   double time_taken;

   time_taken = (end.tv_sec - start.tv_sec) * 1e6;
   time_taken = (time_taken + (end.tv_usec - start.tv_usec)) * 1e-6;
   cout << "Time taken by program is : " << fixed << time_taken <<
setprecision(6);
   cout << " sec" << endl;
   return 0;
}
```

Ömer ÇETİN

4. clock_gettime() Fonksiyonun C & C++'da Kullanımı ile Zaman Alma

- **clock_gettime()** : Clock_gettime () işlevi, clock_id tarafından belirtilen saatin geçerli zamanını alır ve bunu tp ile gösterilen tampon belleğe koyar.
- **Header File** : "time.h".
- **Prototype/ Syntax** : int clock_gettime(clockid_t clock_id, struct timespec *tp);

tp parametresi en az aşağıdaki üyeleri içeren bir yapıya işaret eder:

struct timespec {
time_t tv_sec; //seconds
long tv_nsec; //nanoseconds
};

- **Return Value** : başarı için 0, başarısızlık için -1 döndürür.
- **clock_id** : clock id = CLOCK_REALTIME, CLOCK_PROCESS_CPUTIME_ID, CLOCK_MONOTONIC ... etc.
- **CLOCK_REALTIME** : Gerçek yani duvar saati süresini ölçen saat.
- **CLOCK_PROCESS_CPUTIME_ID** : İşlemciden işlem başına yüksek çözünürlüklü zamanlayıcı.
- **CLOCK_MONOTONIC** : Sistem tarihinden etkilenmeyen yüksek çözünürlüklü zamanlayıcı (örneğin NTP günlükleri).

```
#include <bits/stdc++.h>
#include <sys/time.h>
using namespace std;

// Süresi ölçülmek istenen örnek fonksiyon
void fun() {
   for (int i=0; i<10; i++) {

   }
}

int main() {
   struct timespec start, end;

   // zamanı başlat.
   // clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &start);
   // clock_gettime(CLOCK_REALTIME, &start);
   clock_gettime(CLOCK_MONOTONIC, &start);

   // C ve C ++ 'nın G / Ç işlemleri senkronizasyon iptali.
   ios_base::sync_with_stdio(false);
   fun();
   // zamanı durdur.
   // clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &end);
   // clock_gettime(CLOCK_REALTIME, &end);
   clock_gettime(CLOCK_MONOTONIC, &end);

   // Yürütülme süresini hesapla.
   double time_taken;
   time_taken = (end.tv_sec - start.tv_sec) * 1e9;
   time_taken = (time_taken + (end.tv_nsec - start.tv_nsec)) * 1e-9;

   cout << "Time taken by program is : " << fixed
<< time_taken << setprecision(9);
   cout << " sec" << endl;
   return 0;
}
```


5. chrono::high_resolution_clock ile C++'da Zaman Alma

- chrono: Chrono kütüphanesi tarih ve saatle uğraşmak için kullanılır. Bu kütüphane, zamanlayıcıların ve saatlerin farklı sistemlerde farklı olabileceği ve böylece zaman içinde hassasiyet açısından iyileşme sağlayacak şekilde tasarlanmıştır. chrono bir başlığın adıdır, fakat aynı zamanda bir alt ad alanının da adıdır, Bu başlıktaki tüm öğeler doğrudan std ad alanının altında (standart kitaplığın çoğu gibi) değil, std :: chrono ad alanının altında tanımlanır.

```
#include <bits/stdc++.h>
#include <chrono>
using namespace std;

// Süresi ölçülmek istenen örnek fonksiyon
void fun() {
   for (int i=0; i<10; i++) {

   }
}

int main() {
   auto start = chrono::high_resolution_clock::now();

   // C ve C ++ 'nın G / Ç işlemleri senkronizasyon iptali.
   ios_base::sync_with_stdio(false);

   fun();

   auto end = chrono::high_resolution_clock::now();

   // Yürütülme süresini hesapla.
   double time_taken = chrono::duration_cast<chrono::nanoseconds>(end
- start).count();

   time_taken *= 1e-9;

   cout << "Time taken by program is : " << fixed << time_taken <<
setprecision(9);
   cout << " sec" << endl;
   return 0;
}
```

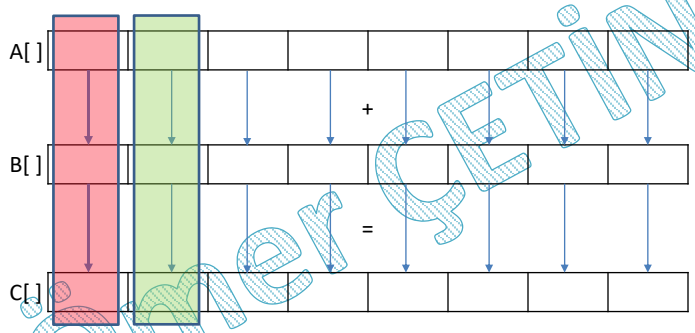

Ömer ÇETİN

OpenMP Uygulama Örnekleri - 1

VEKTÖR TOPLAMA

Ömer ÇETİN

Openmp İle C++ Programlama Dilinde Vektör Toplama



Süreçler bağımsız, veriler arasında bir bağıntı yok. Dolayısıyla **Veri bağımlılıkları** (data dependencies), **veri çatışmaları** (data conflicts), **yarış koşulları** (race conditions) veya **ölümcül kilitlenmeleri** (deadlocks) kontrol etmek sorun değil.

Ömer ÇETİN

Openmp İle C++ Programlama Dilinde Vektör Toplama Örneği - 1

```
#include "stdafx.h"
#include <stdlib.h>   //malloc and free
#include <stdio.h>    //printf
#include <omp.h>      //OpenMP

// Bu basit örnek için çok küçük değerler
#define ARRAY_SIZE 8     //Elementleri toplanacak olan dizilerin büyüklüğü.
#define NUM_THREADS 4    //Vektör toplamak için kullanılacak iplik (thread) sayısı.

int main (int argc, char *argv[])
{ // a ve b dizilerinin öğeleri eklenir ve c dizisine yerleştirilir
    int * a;
    int * b;
    int * c;

    int n = ARRAY_SIZE;                  // dizi elemanlarının sayısı
    int n_per_thread;                    // iplik başına eleman
    int total_threads = NUM_THREADS;     // kullanılacak iplik sayısı
    int i;                               // döngü index değeri

    // diziler için yer ayır
    a = (int *) malloc(sizeof(int)*n);
    b = (int *) malloc(sizeof(int)*n);
    c = (int *) malloc(sizeof(int)*n);
```

Ömer ÇETİN

Openmp İle C++ Programlama Dilinde Vektör Toplama Örneği - 2

```
    // ardışık tamsayı değerlerine sahip a ve b dizilerine ilk değer atama
    // basit bir örnek olarak
    for(i=0; i<n; i++) {
        a[i] = i;
    }
    for(i=0; i<n; i++) {
        b[i] = i;
    }

    // İplik sayısını ayarlamak, sadece örnekleme amacıyla 4'e kodladık.
    omp_set_num_threads(total_threads);

    // Her bir işlemin kaç eleman üzerinde çalışacağını belirleyin
    n_per_thread = n/total_threads;
```

Ömer ÇETİN

Openmp İle C++ Programlama Dilinde Vektör Toplama Örneği - 3

```
    // Vektör eklemesini hesapla
    // Burası 4 ipliğin paralel olarak yürütülmesi için özel olarak 'çatallandığı' (fork) yer.
    // Bu, pragma ifadesi tarafından yönlendirilir ve iplik seviyesinde çatallanma üzerine
    // işlem yükü dağıtılır. Burada 'statik zamanlama' tipini kullanıyoruz, böylece her iş
    // parçacığı orijinal 8-elemanlı dizinin 2 elemanlı bir parçası üzerinde çalışıyor.

    #pragma omp parallel for shared(a, b, c) private(i) schedule(static, n_per_thread)
    for(i=0; i<n; i++) {
        c[i] = a[i]+b[i];
        // Ben hangi ipliğim? Bu küçük örnek için kimin üzerinde çalıştığını gösterin
        printf("Thread %d works on element%d\n", omp_get_thread_num(), i);
    }

    // Doğruluğunu kontrol edin (sadece küçük vektörler için makul)
    // Esasında nihai versiyonda kullanmayacağımız bir test
    printf("i\ta[i]\t+\tb[i]\t=\tc[i]\n");
    for(i=0; i<n; i++) {
        printf("%d\t%d\t\t%d\t\t%d\n", i, a[i], b[i], c[i]);
    }

    // belleği boşalt
    free(a);  free(b); free(c);

    return 0;
}
```


Openmp İle C++ Programlama Dilinde Vektör Toplama Örneği - 4

C:\WINDOWS\system32\cmd.exe

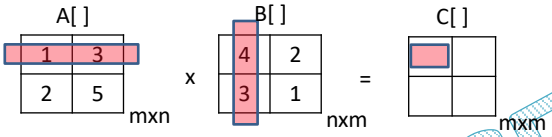
```
Thread 0 works on element0
Thread 1 works on element2
Thread 1 works on element3
Thread 3 works on element6
Thread 3 works on element7
Thread 2 works on element4
Thread 2 works on element5
Thread 0 works on element1
i       a[i]    +       b[i]    =       c[i]
0       0               0               0
1       1               1               2
2       2               2               4
3       3               3               6
4       4               4               8
5       5               5               10
6       6               6               12
7       7               7               14
Press any key to continue . . .
```

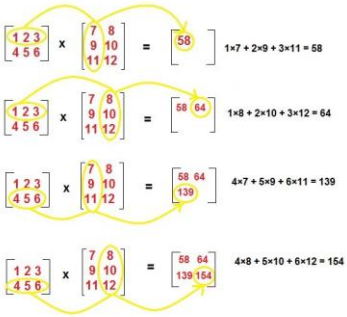
OpenMP Uygulama Örnekleri - 2

MATRİS ÇARPIMI

Ömer ÇETİN

Matris Çarpımı





Süreçler bağımsız, veriler arasında bir bağıntı yok. Dolayısıyla **Veri bağımlılıkları** (data dependencies), **veri çatışmaları** (data conflicts), **yarış koşulları** (race conditions) veya **ölümcül kilitlenmeleri** (deadlocks) kontrol etmek sorun değil.

ÇETİN

Seri Matris Çarpımı

```
for (I=0; I<n; I++)
    for (j=0; j<n; j++)
        c[I][j] = 0;
        for (k=0; k<n; k++)
            c[I][j] = c[I][j] + a[I][k] * b[k][j];
```

Ömer ÇETİN

OpenMP ile Paralel Matris Çarpımı

```
#pragma omp parallel for private(j, k)
for (I=0; I<n; I++)
    for (j=0; j<n; j++)
        c[I][j] = 0;
        for (k=0; k<n; k++)
            c[I][j] = c[I][j] + a[I][k] * b[k][j];
```

Ömer ÇETİN

OpenMP ile Paralel Matris Çarpımı

```
#include "stdafx.h"
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

#define N 4

void multiply(int mat1[][N], int mat2[][N], int res[][N]) {
   int i, j, k;
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++) {
         res[i][j] = 0;
         for (k = 0; k < N; k++)
            res[i][j] += mat1[i][k] * mat2[k][j];
      }
   }
}


void multiply_paralel(int mat1[][N], int mat2[][N], int
res[][N]) {
   int i, j, k;
   #pragma omp parallel for private(j, k)
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++) {
         res[i][j] = 0;
         for (k = 0; k < N; k++)
            res[i][j] += mat1[i][k] * mat2[k][j];
      }
   }
}
```

```
// Driver Code
int main() {
   int i, j;
   int mat1[N][N];
   int mat2[N][N];
   clock_t start,finish,startp,finishp;
   double processing_time,processing_time_p;

   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++)
         mat1[i][j] = (rand() % 100) + 1;
   }

   cout << "Mat1 matris elemanlari: \n";
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++)
         cout << mat1[i][j] << "\t";
      cout << "\n";
   }
   cout << "\n";

   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++) {
         mat2[i][j] = (rand() % 100) + 1;
      }
   }

   cout << "Mat2 matris elemanlari: \n";
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++)
         cout << mat2[i][j] << "\t";
      cout << "\n";
   }
   cout << "\n";
```

Ömer ÇETİN

OpenMP ile Paralel Matris Çarpımı

```
   int res[N][N]; // To store result

   start= clock();
   multiply(mat1, mat2, res);
   finish=clock();
   processing_time=(double)(finish-start)/CLOCKS_PER_SEC;

   cout << "Seri hesaplama sonuc: \n";
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++)
         cout << res[i][j] << "\t";
      cout << "\n";
   }

   startp= clock();
   multiply_paralel(mat1, mat2, res);
   finishp=clock();
   processing_time_p=(double)(finishp-startp)/CLOCKS_PER_SEC;

   cout << "Paralel Hesaplama Sonuc \n";
   for (i = 0; i < N; i++) {
      for (j = 0; j < N; j++)
         cout << res[i][j] << "\t";
      cout << "\n";
   }

   printf("Seri hesaplama %lf \n", processing_time);
   printf("Paralel hesaplama %lf \n", processing_time_p);
   return 0;
}
```

C:\WINDOWS\system32\cmd.exe

```
Mat1 matris elemanlari:
42      68      35      1
70      25      79      59
53      65      6       46
32      28      62      92

Mat2 matris elemanlari:
96      43      28      37
92      5       3       54
93      83      22      17
19      96      48      27

Seri hesaplama sonuc:
13562   5147    2198    5848
17488   15356   6605    6876
13460   7948    4299    7185
17962   17644   8160    8084
Paralel Hesaplama Sonuc
13562   5147    2198    5848
17488   15356   6605    6876
13460   7948    4299    7185
17962   17644   8160    8084
Seri hesaplama 0.000000
Paralel hesaplama 0.000000
Press any key to continue . . .
```

Ömer ÇETİN

OpenMP Performans

```
void multiply_paralel(int mat1[][N],int mat2[][N], int
res[][N])
  {
    int i, j, k;
    #pragma omp parallel for private(j, k)
    for (i = 0; i < N; i++)
    {
      for (j = 0; j < N; j++)
      {
        res[i][j] = 0;
        for (k = 0; k < N; k++)
          res[i][j] += mat1[i][k] * mat2[k][j];
      }
    }
  }
```

Matris Carpımı N = 50
Seri hesaplama 0.000001
Paralel hesaplama 0.001000

Matris Carpımı N = 100
Seri hesaplama 0.005000
Paralel hesaplama 0.004000

Matris Carpımı N = 200
Seri hesaplama 0.044000
Paralel hesaplama 0.016000

Ömer ÇETİN

OpenMP Uygulama Örnekleri – 3

KABARCIK SIRALAMA

Ömer ÇETİN

Paralel Kabarcık Sıralama (Bubble Sort)

Ardışıl (Sıralı) Kabarcık Sıralama Algoritması

N = 6

```
#include <algorithm>
            . . .
for( int i = 0; i < N; i++ )
{
      bool stop = true;

      for( int j = 0; j < N-1; j++ )
      {
            if( B[ j ] > B[ j+1 ] )
            {
                  std::swap( B[ j ], B[ j+1 ] );
                  stop = false;
            }
      }

      if( stop )
            break;
}
```

	Step #				
original	**0**	**1**	**2**	**3**	**4**
6	5	4	3	2	1
5	4	3	2	1	2
4	3	2	1	3	3
3	2	1	4	4	4
2	1	5	5	5	5
1	6	6	6	6	6

Paralel Kabarcık Sıralama (Bubble Sort)

```
#include <algorithm>
            . . .
for( int i = 0; i < N; i++ )
{
      bool  stop = true;

      if( B[ 0 ] > B[ 1 ] )
      {
            std::swap( B[ 0 ], B[ 1 ] );
            stop = false;
      }

      if( B[ 1 ] > B[ 2 ] )
      {
            std::swap( B[ 1 ], B[ 2 ] );
            stop = false;
      }

      if( B[ 2 ] > B[ 3 ] )
      {
            std::swap( B[ 2 ], B[ 3 ] );
            stop = false;
      }

      . . .

      if( stop )
            break;
}
```

Kabarcık sıralamanın bu versiyonu neden paralel koşturulamaz?

İç döngü (j) döngüsünü kaldıralım, böylece for döngüsünün gerçekte neye benzediğini görelim. Bu if-blokların her birinin farklı bir ipliğe atandığını varsayalım (OpenMP'nin farklı for döngü geçişlerini farklı ipliklere atamaya çalıştığını unutmayın).

İpliklere iş parçacığı planlaması üzerinde açık bir kontrolümüz olmadığını hatırlatarak, birinci ve ikinci if-bloklarının hem B[1]'den okuduğunu hem de B[1]'e yazdığını unutmayın. Bunun hangi sırada olduğunu kontrol etmek için senkronizasyon yoktur. **Klasik bir yarış durumumuz var.**

Çözüm, bir geçişte, B [0] ve B [1] 'e tek bir ipliğin erişimine, B [2] ve B [3]' e başka bir ipliğin erişimine, B [4] ve B [5]'e başka bir ipliğin erişimine vb. izin verir.

Sonra, bir sonraki geçişte, B[1] ve B[2]'ye tek bir ipliğin erişimine, B[3] ve B[4]' e başka ipliğin erişimine, B[5] ve B[6] 'ya başka ipliğin vb. erişimine izin vermek.

ÖMER ÇETİN

Paralel Kabarcık Sıralama (Bubble Sort)

Paralel Koşturmaya Uygun Düzenlenmiş Kabarcık Sıralama Algoritması

N = 6

```
#include <algorithm>
        . . .
for(  int i = 0;  i < N;  i++ )
{
      int first = i % 2;      // 0 if i is 0, 2, 4, ...
                              // 1 if i is 1, 3, 5, ...

      #pragma omp parallel for default(none),shared(A,first)

      for(  int j = first;  j < N-1;  j += 2  )
      {
            if(  A[ j ]  >  A[ j+1 ]  )
            {
                  std::swap(  A[ j ],  A[ j+1 ]  );
            }
      }
}
```

original	Step # 0	1	2	3	4	5
6	5	5	3	3	1	1
5	6	3	5	1	3	2
4	3	6	1	5	2	3
3	4	1	6	2	5	4
2	1	4	2	6	4	5
1	2	2	4	4	6	6

Paralel Kabarcık Sıralama (Bubble Sort)

Karşılaştırma

N = 6

original	Step # 0	1	2	3	4
6	5	4	3	2	1
5	4	3	2	1	2
4	3	2	1	3	3
3	2	1	4	4	4
2	1	5	5	5	5
1	6	6	6	6	6

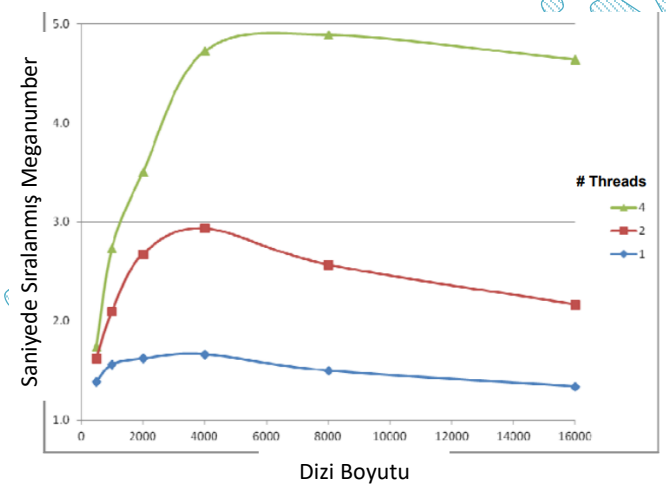
Klasik Algoritma

original	Step # 0	1	2	3	4	5
6	5	5	3	3	1	1
5	6	3	5	1	3	2
4	3	6	1	5	2	3
3	4	1	6	2	5	4
2	1	4	2	6	4	5
1	2	2	4	4	6	6

Düzenlenmiş Algoritma

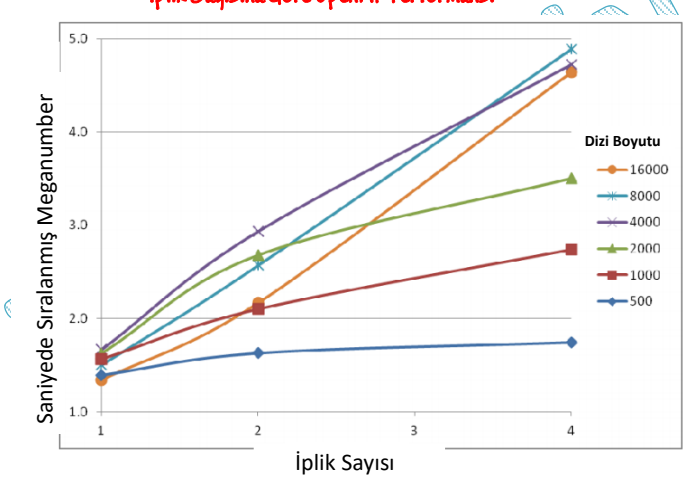
Paralel Kabarcık Sıralama (Bubble Sort)

Dizi Boyutu İşlevi Olarak OpenMP Performansı



Paralel Kabarcık Sıralama (Bubble Sort)

İplik Sayısına Göre OpenMP Performansı



OpenMP Uygulama Örnekleri - 4

GEZGİN SATICI

Ömer ÇETİN

Gezgin Satıcı Problemi (TSP)

- **Harita,** şehirleri temsil eden düğümler ve şehirler arasındaki mesafeleri temsil eden kenarlarla bir grafik olarak gösterilir.
- Özel bir **düğüm** (şehir), turun başlangıç noktasıdır.
- **Gezgin satıcı problemi** (Travelling Salesman Problem - TSP), en küçük mesafedeki tüm düğümleri kapsayan rotayı her şehre bir kere uğrayarak bulmayı amaçlar.
- Bu, bilinen bir **NP-complete** problemidir.

Ömer ÇETİN

Seri (Sequential) TSP

```
Init_q();
init_best();
While ((p = dequeue()) != NULL) {
    for each expansion by one city {
        q = addcity (p);
        if (complete(q)) {update_best(q);}
        else enqueue(q);
    }
}
```

Ömer ÇETİN

OpenMP TSP

```
Do_work() {
  While ((p = dequeue()) != NULL) {
    for each expansion by one city {
        q = addcity (p);
        if (complete(q)) {update_best(q);}
        else enqueue(q);
    }
  }
}

main() {
   init_q();
   init_best();
   #pragma omp parallel for
   for (i=0; I < NPROCS; i++)
      do_work();
}
```

Ömer ÇETİN

OpenMP Sonuç

Özet:

- OpenMP, paylaşımlı bellek programlama için **kompakt**, ancak **güçlü** bir programlama modeli sunar.
  - Paralel programlar oluşturmak için OpenMP'yi kullanmak çok kolaydır.
- OpenMP programın **sıralı sürümünü** korur.
- **Bir OpenMP programı geliştirme adımları:**
  1. Sıralı bir programdan başlayın.
  2. Fazla zaman harcayan kod bölümü tespit edin / belirleyin.
  3. Önemli döngülerin paralelleştirilip paralelleştirilemeyeceğini belirleyin.
     - Döngülerin kritik bölümleri, indirgeme (reduction) değişkenleri vb. olabilir.
  4. Paylaşılan ve özel değişkenleri belirleyin.
  5. OpenMP kodlarını ekleyin.

Ömer ÇETİN

OpenMP Sonuç

- Kullanım kolaylığı
  - OpenMP iplik bakımı ile ilgilenir.
    - Pthread üzerinde büyük gelişme.
- Senkronizasyon
  - Çok daha yüksek yapılar (kritik bölüm, bariyer).
  - Pthread üzerinde büyük gelişme.

OpenMP kullanımı kolaydır !!

Ömer ÇETİN

OpenMP Sonuç

- **Etkileyicilik**
  - Veri paralelizasyonu:
    - SIMD
    - Paradigmada çok iyi uyuyor
  - Görev paralelliği:
    - MISD (TSP örneği)
    - Yeni thread'ler oluşturmak için OpenMP yapılarını kullanın. OpenMP, pthread'den çok farklı değildir.

Ömer ÇETİN

OpenMP Sonuç

- Mimari özellikleri açığa çıkarmak (performans):
  - pthread yaklaşımına benzer
    - Varsayım: işi threadlere paylaştırarak = geliştirilmiş performans.
    - Bu varsayım gerçekte ne kadar geçerlidir?
      - Genel giderler, çekişmeler, senkronizasyon vb.
  - Bu, OpenMP için zayıf bir nokta: OpenMP programının performansının anlaşılması biraz zor.

Ömer ÇETİN

OpenMP Sonuç

- OpenMP ile ilgili temel sorunlar: **performans**
  - Bunu çözmenin açık bir yolu var mı?
    - Daha fazla mimari özellik sunuyor musunuz?
  - Performans sorunu, paralel program yazmamızın esas amacı değil mi?
    - OpenMP programları sıralı programlar ile başlar.
    - Sorunu gerçekten çözmek için verimli paralel programlar yazmak için yeni bir yol bulmamız gerekebilir.

Ömer ÇETİN

OpenMP Performans

```
int main(int argc, char* argv[])
{
    omp_set_num_threads(8);
    CStopWatch elapsed;
    elapsed.startTimer();
    float times=0.0f;
    CStopWatch stopWatch;
```

```
#pragma omp parallel for
  for (int loop=0;loop < loopcount;loop++) {
    stopWatch.startTimer();
    doCalcs();
    stopWatch.stopTimer();
    if (loop>0) {
      times += (float)stopWatch.getElapsedTime();
      printf( "time = %f",stopWatch.getElapsedTime() );
    }
  }
```

```
  elapsed.stopTimer();
printf("Average run time = /f" , times/(loopcount-1);
Printf("Total time = /f" , elapsed.getElapsedTime());
}
```

Ömer ÇETİN

OpenMP Performans

```
#include <omp.h>
#include <stdio.h>
#include <time.h>

int main()
{
        const int size=240000;
        int sonTable[size];
        clock_t start,finish,start2,finish2;
        double processing_time,processing_time2;

        start= clock();
        for (int n=0; n<size; ++n)
        {
                sonTable[n]=n*2;
                printf ("Table %d elemani =%d from thread %d, nthreads %d \n", n, sonTable[n], omp_get_thread_num(), omp_get_num_threads());
        }
        finish=clock();
        processing_time=(double)(finish-start)/CLOCKS_PER_SEC;

        start2= clock();
        omp_set_num_threads(8);
        #pragma omp parallel for
        for (int n=0; n<size; ++n)
        {
                sonTable[n]=n*2;
                printf ("Table %d elemani =%d from thread %d, nthreads %d \n", n, sonTable[n], omp_get_thread_num(), omp_get_num_threads());
        }
        finish2=clock();
        processing_time2=(double)(finish2-start2)/CLOCKS_PER_SEC;

        printf("Paralel hesaplama zamanı %lf \n", processing_time2);
        printf("Seri hesaplama %lf \n", processing_time);
}
```

Sıralı Hesaplama

Paralel Hesaplama

Ömer ÇETİN

ÖDEV

Literatürde yer alan IPC problemlerini OpenMP ile kodlayarak nasıl yaptığınızı açıklayan bir sunum hazırlayınız... (Çalışan kod ve sunum birlikte gönderilecektir.)

1. Okuyucu – Yazıcı
2. Üretici – Tüketici
3. Makarna Yiyen Düşünürler
4. Uyuyan Berber

Ömer ÇETİN

Fazlası

- https://www.openmp.org/
- http://jakascorner.com/blog/2016/04/omp-introduction.html
- https://computing.llnl.gov/tutorials/openMP/
- https://www.slac.stanford.edu/comp/unix/farm/openmp.html

Ömer ÇETİN